

TP 3 - Architecture logicielle

Objectifs du TP :

- Choisir et réaliser une architecture adaptée au besoin client

Enoncé

À l'attention de l'équipe de développement,

Notre plateforme de trading de crypto-monnaies actuelle, Ynov Exchange, rencontre de graves problèmes de fiabilité. Lors des pics de volatilité du Bitcoin (BTC), notre serveur unique sature, et pire encore, nous avons constaté des erreurs de solde : certains clients sont débités en Euros (EUR) sans jamais recevoir leurs Bitcoins, car la base de données plante au milieu de la transaction.

Votre Mission :

Nous avons décidé de repartir de zéro. Nous voulons une nouvelle infrastructure capable de gérer l'achat de BTC via un compte en EUR.

Vous avez carte blanche sur l'organisation technique, mais la direction impose des garanties strictes :

1. Fiabilité Totale : Il est strictement interdit qu'un client perde son argent si le stock de Bitcoins est vide ou si un serveur tombe en panne au mauvais moment.
2. Indépendance : Nous voulons pouvoir mettre à jour le système de gestion des portefeuilles sans risquer de casser le catalogue des actifs ou le système de commande.
3. Traçabilité : En cas de litige avec un client, nous devons être capables de voir exactement par quel chemin est passée sa commande et quel service a éventuellement échoué.
4. Interface de Test : Nos partenaires doivent pouvoir comprendre et tester vos points d'entrée sans avoir à lire tout votre code source.

Les Besoins du Front-end

L'équipe Web a besoin d'une interface unique pour :

- Afficher le solde de l'utilisateur (EUR et BTC).
- Lancer un ordre d'achat.
- Consulter l'historique et l'état des commandes (qui ne sont plus instantanées, mais traitées en arrière-plan).

Contraintes de rendu

Vous devez nous fournir un environnement complet (utilisant Docker pour les outils tiers comme le broker de messages).

Votre projet sera évalué sur votre capacité à découper intelligemment les responsabilités, à faire communiquer vos briques de manière moderne (asynchrone quand c'est nécessaire) et à assurer la cohérence globale des données sans utiliser de "jointures" impossibles entre bases de données.

Note aux architectes : "Le succès de ce projet dépend de votre capacité à anticiper les pannes. Un bon système n'est pas un système qui ne plante jamais, c'est un système qui sait s'auto-réparer ou s'annuler proprement lorsqu'un incident survient."

Liste des fonctionnalités attendues

- **Découpage par domaines métier** : Séparation stricte des responsabilités (ex: gestion des fonds EUR, gestion du stock BTC, gestion des ordres) via des micro-services indépendants possédant chacun leur propre base de données.
- **Flux transactionnel asynchrone** : Implémentation d'un système de messagerie (Message Broker) pour traiter les achats sans bloquer l'utilisateur et garantir la livraison des messages.
- **Gestion de l'intégrité (Saga Pattern)** : Mise en place d'une logique de compensation (annulation/remboursement) automatique si une étape de l'achat échoue (ex: stock BTC épuisé après débit EUR).
- **Point d'entrée unifié** : Mise en place d'une API Gateway pour rediriger les requêtes du front-end vers les services concernés et centraliser la sécurité.
- **Auto-documentation** : Génération d'une documentation interactive (Swagger/OpenAPI) pour chaque point d'accès afin de permettre aux partenaires de tester l'API.
- **Traçabilité distribuée** : Attribution d'un "Correlation-ID" unique à chaque requête client pour suivre son parcours complet à travers les différents services dans les logs.
- **Surveillance de l'état (Health Checks)** : Exposition d'une route de diagnostic par service pour vérifier en temps réel la santé des briques (connexion BDD, connexion Kafka).
- **Tableau de bord** : Affichage dynamique des soldes (Euros et Bitcoins) récupérés depuis les services sources.
- **Passage d'ordre** : Formulaire permettant de déclencher une demande d'achat de BTC.
- **Suivi en temps réel** : Affichage du statut de la commande (ex: "En cours", "Validée", "Échouée") basé sur le traitement asynchrone du backend.

Conseils :

- Commencez par dessiner le diagramme de séquence de votre achat : que se passe-t-il si le service de stockage est indisponible ?...
- Ne cherchez pas à faire des jointures SQL entre vos services, utilisez les événements pour synchroniser les informations nécessaires.
- Le front-end n'est pas du tout la partie la plus importante du projet en termes de notation , ne vous focalisez pas dessus en priorité.

Bonus :

- Mise en place d'une authentification distribuée
- Travail sur le front-end supplémentaire